Page Checks — Technical Reference

An engineering-oriented reference for the checks in **Is It Agent-Ready? for SitecoreAI**: the scoring model, every check with its thresholds and credit, and where each one lives in the code. The Page Builder panel ("Is this page agent-ready?") and the dashboard ("Is this site agent-ready?") use **the same engine**, so the same inputs give the same score in both. The panel also scores the Page Builder **draft**, so its score follows unpublished edits (see "Draft and live" below). Dashboard specifics (roll-up, cache, site checks card) are in `PAGE-MATRIX-TECHNICAL.md`.

Source of truth: `src/lib/checks/` — `registry.ts` (the check list), `page/*.ts` and `site/*.ts` (the checks), `scoring.ts` (pool maths), `gate.ts`, `engine.ts` (entry points), `context.ts` (inputs), `draft-diff.ts` (draft vs live comparison), `json-ld.ts` (shared JSON-LD parser), `view-model.ts` (what the UI shows). HTML parsing is in `src/lib/html/`. Plan reference: `docs/IS-IT-AGENT-READY-PLAN.md` sections 5 and 7.

Scoring model

- Every check belongs to one of **four categories**, phrased as questions: **Reach** (Can AI agents get to it?), **Read** (Can they read it?), **Understand** (Do they understand it?), **Trust** (Will they trust and cite it?).
- Every check has a **tier**. **Essential** checks share an **80-point pool**, **Recommended** checks a **20-point pool**, and **Bonus** checks add **up to +5**. The total is capped at 100.
- A check returns a **credit from 0 to 1** (partial credit is normal) and a status (`pass` / `warning` / `fail` / `info`). Its points are `credit × its share of the tier pool`. Shares are equal unless a check sets the optional `weight` multiplier (default 1; none are set today).
- A check can also return **N/A** (`not-applicable`) or **not checked** (`not-checked`: AI pending, PageSpeed not run, fetch failed). Both are **left out of the pools, never failed**. The pool is split across the checks that are scored, so a page is only graded on what applies to it.
- If Essential or Recommended has nothing scored, the other one is scaled up so the base still totals 100. With nothing scored at all, there is no score yet (`null`).
- The score is rounded to a whole number. Exact points are kept per check and per category (`ScoreBreakdown` in `types.ts`).
- Thresholds are shared by the panel and the dashboard (`src/lib/score-thresholds.ts`): **Agent-ready ≥ 80**, **Almost there 60–79**, **Not ready < 60**.

Points. Six Essential page checks exist (P2, P3, P5, P8, P9, P10). In the panel, when all apply, each is worth about 13.3 points. On the dashboard P3 isn't checked (no draft), so the other five are worth 16 each. On the home page P10 is usually N/A. The Recommended checks share 20 points (about 1.5 each when all thirteen apply; P6 is N/A until the page serves Markdown). So a site that doesn't serve Markdown loses about 13 points per page in the panel and 16 on the dashboard (the demo page: 91 → 74 on live HTML).

Scopes. Page checks run per page in the panel and in dashboard scans. Site checks run once per site and get **their own site score**; they are never part of a page score. There are twelve, S1–S12 (see "Site checks").

Same inputs, same score. Both views build the check context with `buildPageContext` (`context.ts`) from the same inputs: the page's live HTML (`/api/preview-fetch` → `ParsedHtml`), the AI results (`/api/ai-check`, cached by content hash), and the same Sitecore fields read by the same names (`SCORING_FIELD_NAMES`: title, meta description, `robotsMetaTag`, `__Updated`). The panel reads them with `pageFieldsFromFieldMap`, the dashboard with `pageFieldsFromValues`. `src/lib/checks/parity.test.ts` covers this.

Where the panel and the dashboard differ, by design:

- **The draft.** The panel's body checks and AI checks read the Page Builder draft. It includes unpublished edits and content rendered by JavaScript (e.g. BrandX's Related Articles cards), so the panel's score can differ from the dashboard's even for a page with no edits. Agreed on 2026-11-01 (plan section 7, Phase 4).
- **P3** needs the draft, so it's only scored in the panel. The dashboard shows it as "Page Builder only" (not checked, left out of the pools).
- **PageSpeed:** only the panel can run it, so after you run it there, the panel includes P7 and the dashboard doesn't.

Draft and live (plan section 7)

Data	Source	Read by
Draft: `client.getPageHTML()`, sanitised and parsed in the browser	`usePageHtml` (`src/hooks/usePageHtml.ts`), panel only	Body checks: P3, P4, P8, P12, P13, P14, P15, P19, P20. AI body: P16, P21 and the meta generator
Live: `/api/preview-fetch` (server HTML, with the editing secret)	`useScan` (panel), `useSitePages` (dashboard)	Head checks: P2, P9 (fallback), P10, P11, P17, P18, the gate, S7. P3's comparison
Fields: authoring GraphQL	`usePageFields`	P9, P18, P6, the gate
Markdown: `/api/markdown-check` (the published page as an agent and as a browser)	`useMarkdownCheck` (panel), `useSitePages` (dashboard)	P5, P6. The Agent View's preview is converted from the draft (`DraftHtml.source`, the sanitised HTML); its Before view breaks down the published HTML (`html.anatomy`, `html.excerpt`)

- **Context.** `PageInput.draft` is optional: the dashboard leaves it out. `buildPageContext` sets `body` to the draft's `ParsedHtml` when there is one, otherwise the live one, and `bodySource` to `"draft"` or `"live"`. Body checks read `ctx.body`; head checks read `ctx.html`.
- **P10 stays fully live.** Its JSON-LD (only in the `<head>`, which the draft lacks) and its Q&A pairs come from the same render.

- **Source chip.** A check declares `sources` (body checks declare `"draft"`). `resolveSources` (`engine.ts`) turns `"draft"` into the body's actual source and adds it to AI checks, and every `CheckOutcome` carries the result as `sources`. The panel shows it as a chip next to each check's title (Draft / Live / Fields / AI / PageSpeed; "Draft vs Live" for P3), with the full wording when the row is open.
- **Refresh.** `usePageHtml` reads the draft 1.5 s after a page change (so the canvas can load the new page) and 1.5 s after the last `pages.content.fieldsUpdated` (`usePageFields.contentVersion`), and at once on **Re-scan page**. Only the latest read may set state. Each read takes about 30–50 ms. Console: `[draft] getPageHTML: N chars …; removed {…}`.
- **Retry while the canvas loads.** Right after a page opens, Page Builder answers `getPageHTML` with `CoreError: [client SDK] [CANVAS BUG] Service implementation is not provided` until its canvas has loaded (seen on production on 2026-11-02; a Re-scan a moment later worked). So while a page has no draft yet, a failed read is retried after 2, 4 and 8 s (`readWithRetry`, about 15 s in all). The status stays `loading` meanwhile, so rows show "Scanning…" and the AI checks wait for the draft. Console: `[draft] getPageHTML not ready (attempt N); trying again in N s`. A newer read (page change, field update, Re-scan) stops the retries.
- **Fallback.** If every attempt fails, the status is `unavailable` and the body checks read live, as before: chips say Live, and P3 says "Draft not available … Re-scan to try again" (`draftUnavailable` in the context; the dashboard's "Page Builder only" is unchanged). A failed refresh of a page that already has a draft isn't retried and keeps that draft, so the score doesn't jump mid-edit.
- **Unpublished pages.** If preview-fetch fails (no published page yet), the head checks are not checked and the body checks still score from the draft.
- **AI checks.** They judge the body text (`useScan.bodyText`: the draft's `mainText`, or live's without a draft). They run once per page load and per Re-scan, on the first ready body, never on every edit. When the text has changed since, the AI rows say "Based on an earlier version of the text" with a **Re-run** link (`useScan.aiStale`). Results are cached by content hash as before.
- **"Draft differs from live" banner** (`src/components/panel/DraftBanner.tsx`: "Scored from your draft. It differs from the live page …"). `draftDiffersFromLive` (`draft-diff.ts`) compares the draft's and live's `mainText` line by line. It shows when a live line is missing from the draft, or a draft line is an edit of a live one (word overlap ≥ 0.5). Lines only in the draft, with no live counterpart, don't count: they're usually client-rendered, and would otherwise show the banner on every page with a client-fetched listing.

The sanitiser (`src/lib/html/sanitise-draft.ts`)

Every capture goes through `sanitiseDraftHtml` before it's parsed, so no check sees raw editor output. It removes (SPIKE-FINDINGS §3, item 5, and §3.1 for the captures it's tested on):

- **Page Builder help text** (`.ee-help-text-block`, seen in all three BrandX captures):

- "Placeholder: custom-headless-hero"-style labels around each placeholder go.

- A `Fallback: …` **hint** next to the field's value goes. In the FAQ h1, `<span>Frequently Asked Questions</span><div class="undefined ee-help-text-block">Fallback:<!-- --> <span>Frequently Asked Questions</span></div>` becomes one "Frequently Asked Questions".

- When the field is empty, the published page shows the fallback, so the hint is replaced by its value without the label, and the heading keeps its text.

- `[No text in field]` placeholders. Empty headings and paragraphs are then dropped by the extractor.
- **Empty fields' placeholder elements** (class `scEmpty*`), removed whole. The St. Patrick's Day hero has an empty image field, which the editor shows as a grey picture with `alt=""`; left in, P15 would count an image without alt text that the published page doesn't have. The class is read from the raw attributes, because that `<img>`'s `src` is an inline SVG containing `class="st3"`, which node-html-parser takes as the element's class.
- **Empty-state messages** (`isEmptyStateMessage`: "No articles found", "No results", "Nothing to show", "Loading…"). They're returned as `emptyStates` for P3's note.
- `contenteditable` and the editor-only classes `placeholder` and `editing-mode`. `data-component` and `data-icon` are the site's own and stay.

`{{…}}` template tokens (the footer's `© {{year}}`) are kept: the published server HTML has the value (`© 2026`), and the comparison matches a token to any one word.

The gate — P1 `reach.indexable`

Location: `gate.ts`

Runs before any check. The page is **blocked** when:

- the Sitecore **Robots Meta Tag** field (`robotsMetaTag`) contains `noindex`, **or**
- the rendered `<meta name="robots">` or `<meta name="googlebot">` contains `noindex`, `nosnippet` or `max-snippet:0`.

A blocked page shows the gate message instead of a score (`PageReport.score` is `null`, no outcomes). The dashboard reads the same field (it was added to its page query), shows the row as "Blocked (noindex)", and leaves it out of the averages.

Page checks

Body checks read the **draft** in the panel and the **live HTML** elsewhere; head checks always read the live HTML (table above). "Not yet scanned" (not checked) means the HTML hasn't arrived yet.

Reach

ID	Title	Tier	Credit	Was
P2 `reach.canonical`	Canonical points to this page	Essential	self-referencing `<link rel="canonical">` → 1 · points elsewhere → 0.4 · missing → 0	`seo-canonical`

Read

ID	Title	Tier	Credit	Was	
P3 `read.no-js-content`	Content is there without JavaScript	Essential	share of the draft's words in the server HTML: ≥ 95% → 1 · ≥ 80% → 0.5 · less → 0. **Not checked** without a draft (dashboard, fallback) or without the live page	new	
P4 `read.landmarks`	Page structure is machine-readable	Recommended	landmarks present ÷ 4 (`<main>` , `<article>` , `<nav>` , `<header>`); unlabelled buttons/links are listed	`cq-agentic-readiness`	
P5 `read.markdown-page`	Agents can get this page as Markdown	Essential	the accept.md checks, graded like S3: no Markdown for `Accept: text/markdown` → 0; otherwise 0.5 + ⅙ each for HTML for browsers, `Vary: Accept` , Markdown smaller than the HTML. **Not checked** until `/api/markdown-check` is in, when neither request answered, or when the HTML request isn't 200 (unpublished page)	new	
P6 `read.markdown-quality`	The Markdown version is complete	Recommended	⅙ front matter title matches the title field (or `<title>` ; containment either way, so "Pricing \	BrandX" matches "Pricing") · ⅙ front matter description matches the Meta description field (an empty field is a miss) · ⅓ coverage ≥ 90% (½ at 75%) · ⅓ chrome < 25% (½ under 40%). **N/A** until P5 finds Markdown	new
P7 `read.speed`	Page loads fast	Recommended	PageSpeed ≥ 70 → 1 · 50–69 → 0.6 · < 50 or not crawlable → 0. **Not checked** until PageSpeed runs, or when PSI errors	`cq-page-experience`	

P5 and P6 in detail.

- `/api/markdown-check?url=` (`src/lib/server/markdown-check.ts`, `collectMarkdownCheck`) requests the page twice in parallel through `safeFetch`: with `Accept: text/markdown, text/html;q=0.5` (`MARKDOWN_ACCEPT`, the header S3 sends) and with `Accept: text/html`. 10 s each, 5 MB cap, redirects followed (so a Sitecore redirect shows as the final URL). It returns, per request, the status, final URL, `Content-Type`, `Vary`, bytes and estimated tokens (characters ÷ 4, `estimateTokens`: free and deterministic, so the dashboard can run it on every page), plus the Markdown body (the first 50,000 characters when it's `text/markdown`, otherwise 2,000, enough to tell Markdown from HTML). The HTML body comes back only as its breakdown (below): `html.anatomy` always, and `html.excerpt` with `&excerpt=1`, which only the panel sends (the dashboard calls the route for every page and never shows it). Raw evidence only; the engine grades it.
- **The breakdown** (Phase 13, `analyseHtml`, `src/lib/markdown/anatomy.ts`) runs when the HTML request returned a page (200, `text/html`), with or without Markdown. `classifyHtml` gives every character of the HTML one category, walking the `node-html-parser` tree by its source ranges: **content** is the text `extractMainText` reads (the same `mainTextScope` and `isHiddenText` from `extract-html.ts`: `<main>`, or the body without header, nav and footer; hidden elements skipped except toggle-controlled panels, so collapsed FAQ answers count), plus the `<title>` (it becomes the front matter); **scripts** are whole `<script>` elements (`__NEXT_DATA__` and other inline data included); **structured data** is `<script type="application/ld+json">`; **styles** are `<style>` elements and `style=""` attributes; **chrome** is any other text in `<body>` (menus, header, footer, hidden text); **markup** is everything else: tags, attributes, comments, whitespace between tags, and whole `<svg>`, `<noscript>`, `<template>` and `<iframe>` elements. `html.anatomy.tokens` is characters ÷ 4 per category, rounded by largest remainder so the six add up to `html.tokens` exactly; `contentStartsAt` is the token offset of the first content character in `<body>`. `html.excerpt` is the first 16,384 characters (`EXCERPT_MAX_CHARS`) as `{ kind, text }` runs, adjacent runs of one kind merged, with its tokens and `truncated`. Measured on the BrandX replica (2026-11-04): home 53,673 tokens, about 61% markup (Tailwind classes, image `srcSet`s, inline SVG), 38% scripts (the Next.js payload), 0.8% content; the first content at about token 7,500, beyond the excerpt.
- **The preview.** When no Markdown comes back, the route converts the published HTML (`toAgentMarkdown`, `src/lib/markdown/convert.ts`, `node-html-markdown` 2.0.0, pinned) and returns it with its token count. The conversion keeps `<main>` (or the body without header, nav and footer), drops scripts, styles, SVG, iframes and hidden elements (collapsed FAQ panels that a toggle controls stay, as in `extractMainText`), makes links and images absolute, and starts with front matter: `title`, `description` (from the head, or the Sitecore fields in the panel) and `url`. The panel converts the sanitised draft instead when it has one, so the preview includes unpublished edits.
- **P5** grades the two responses with `gradeNegotiation` (`markdown-negotiation.ts`), the grader S3 uses. Its message gives the estimated saving when the page doesn't serve Markdown ("Converted, it would be about 97% fewer tokens"), and its fix lists the options (`MARKDOWN_FIX_SUMMARY`, `src/lib/markdown/fixes.ts`).
- **P6** (`gradeMarkdownQuality`, `src/lib/markdown/quality.ts`) compares the served Markdown with the same page's live HTML. Text is compared normalised (lower case, accents and punctuation removed), Markdown link and image syntax reduced to its text. **Coverage** counts the live page's headings (only those in the main text, so a footer's "Legal" doesn't count), paragraphs and FAQ answers by words: an item is covered when its text appears in the Markdown, or (6+ words) when 90% of its words do. **Chrome** is the share of

Markdown lines that are body text from outside the main content (body blocks that aren't in `mainText`, the headings, paragraphs or Q&A). Front matter is read from a leading `---` block (`key: value` lines, quoted values unquoted).

- **On the BrandX replica** (2026-11-02, before accept-md): every page returns `text/html` for `Accept: text/markdown`, no `Vary`; the home page and `/services` are about 214 KB of HTML (≈ 53,700 tokens), `/contact-and-support` 171 KB; the preview is 600–1,370 tokens, so 97–99% fewer. The preview keeps the breadcrumb (it's inside `<main>`), and some card text runs together where the site's inline elements have no space between them.
- **On the BrandX replica with accept-md** (2026-11-02, after three hand fixes, SPIKE-FINDINGS §8): `/`, `/services` and `/contact-and-support` serve `text/markdown` with `Vary: Accept` and HTML for browsers, 97–98% smaller (home: 214,718 → 6,045 bytes), so P5 is 4 of 4 and S3 passes. P6 on the home page gets full credit: front matter matches, 100% of the main content, 5% menus, header and footer. accept-md keeps the site header (logo, menu, country selector) and relative links.

P3 in detail (`compareBlocks` in `draft-diff.ts`). The draft includes content rendered by JavaScript and the server HTML doesn't (SPIKE-FINDINGS §3, item 4), so the gap is what crawlers that don't run JavaScript miss. Both sides are `ParsedHtml.bodyBlocks`: every text block of `<body>`, header, nav, footer and hidden elements included, each with its nearest `data-component` (field and layout wrappers such as `helpers/fieldwrappers/richtextwrapper` are skipped, so a card's summary belongs to `relatedarticles`), capped at 50,000 chars.

- Text is compared lower-cased, with curly quotes and dashes normalised. A `{{…}}` token matches any one word. A draft block found anywhere in the joined live text is server-rendered, so blocks split differently on each side still match.
- A draft block of **4+ words** that isn't found is either an **edit** (word overlap ≥ 0.5 with a live block missing from the draft: counts as server content) or **client-only**. Shorter blocks (dates, "Read more", labels) aren't judged. Empty-state messages aren't counted and are listed as "Couldn't render in the editor".
- Text on the live page but not in the draft (the St. Patrick's Day paragraph) is never reported by P3.
- Flagged items group the client-only text by component.
- **On BrandX** (captures of 2026-11-01): the Banking article scores half credit, "44 of 522 words need JavaScript (91% in the server HTML)", flagged as `relatedarticles: 44 words` (the cards' titles and summaries). The published page's Related Articles section has only its heading and a loading spinner. St. Patrick's Day passes: its article listing is an empty field in the editor and adds no text.
- **Limitation:** text alone can't tell client-rendered content from new text that isn't published yet, so a brand-new unpublished component shows up as client-only until it's published. The result says so.

Understand

ID	Title	Tier	Credit	Was
P8 `understand.headings`	One clear H1 and logical headings	Essential	half for exactly one H1 (1/6 for several), half for no skipped levels	`seo-headings`
P9 `understand.title-meta`	Title and description are set	Essential	average of title (50–60 chars → 1 · 30–70 → 0.6 · other → 0.25 · missing → 0) and description (120–160 → 1 · other → 0.5 · missing → 0). Reads the Sitecore field first, the rendered tag as fallback	`seo-meta-title` + `seo-meta-description`
P10 `understand.json-ld`	Structured data is present and valid	Essential	mean over the schemas this page needs: valid → 1 · present but missing required properties → 0.5 · missing → 0. A block that doesn't parse caps it at 0.5 (fail). N/A when nothing is expected	`aeo-structured-data` + `aeo-faq-schema`
P11 `understand.metadata`	Page identity metadata is complete	Recommended	present ÷ 5 of `html lang`, `og:type`, `og:title`, `og:description`, `og:image`	`seo-og-tags`
P12 `understand.faq-quality`	FAQ answers are quotable	Recommended	answers of 40–300 chars ÷ all answers. N/A without Q&A pairs	`aeo-faq-answer-quality`
P13 `understand.question-headings`	Sections answer real questions	Recommended	≥ 3 question H2s or ≥ 50% of H2s → 1 · at least one → 0.5 · none → 0. N/A without H2s	`aeo-question-headings` (was off)
P14 `understand.readability`	Easy to read and quote	Recommended	half: share of 20+-word paragraphs at ARI grade ≤ 13; half: average paragraph 40–120 words → 1 · 20–200 → 0.5. N/A without paragraphs	`cq-reading-grade` + the matrix's paragraph length
P15 `understand.image-alt`	Images are described	Recommended	images with alt ÷ all images. N/A without images	`seo-image-alt`

P10 in detail (fixes D2). The three old schema checks always passed and only printed a snippet. P10 reads the real `application/ld+json` blocks through the shared parser and expects:

- **BreadcrumbList** on every page except the home page. Required: `itemListElement` with `position` and `name` on each item.

- **FAQPage** when the page has two or more Q&A pairs. Required: `mainEntity` with `name` and `acceptedAnswer.text` on each question.
- **Article** (or any subtype, e.g. `BlogPosting`, `NewsArticle`) on article templates (template name "Article Detail Page"). Required: `headline` (or `name`).

The fix snippet contains only the missing or incomplete schemas (`buildBreadcrumbJsonLd`, `buildFaqPageJsonLd`, `buildArticleJsonLd` in `helpers.ts`). The site's Organization schema is S7, not part of P10.

Q&A pairs (P10, P12). `extractQaPairs` (`src/lib/html/extract-html.ts`) finds them in **any component**: an accordion toggle (`[aria-controls][aria-expanded]`) whose controlled panel holds the answer (collapsed, `aria-hidden` panels included), or a `<details>` whose `<summary>` is the question. Only toggles that read as a question (end in "?" or start with a question word) count, so menus and "Expand all" are skipped. On BrandX's FAQ page it finds all 8 questions. The AccordionItem datasource fetch in `usePageFields` is no longer used for scoring.

Trust

ID	Title	Tier	Credit	Was
P16 `trust.uniqueness`	Content is original, not generic	Recommended	AI rating `unique` → 1 · `borderline` → 0.5 · `commodity` → 0. Not checked until the AI has run	`cq-uniqueness`
P17 `trust.author`	Author is named and linked	Recommended	named and linked → 1 · named only → 0.5 · none → 0. **Article templates only** (N/A elsewhere)	`cq-author` (was off)
P18 `trust.freshness`	Content is recently updated	Recommended	≤ 180 days → 1 · ≤ 365 → 0.6 · ≤ 730 → 0.25 · older or no date → 0	`cq-freshness` (was off) + the matrix's bands
P19 `trust.depth`	Enough depth to be cited	Recommended	words in body paragraphs: ≥ 800 → 1 · ≥ 400 → 2/3 · ≥ 150 → 1/3 · fewer → 0	the matrix's content depth
P20 `trust.anchor-text`	Links say where they go	Recommended	generic anchors ≤ 20% → 1 · ≤ 40% → 0.5 · more → 0. N/A without links	`seo-anchor-text` (was info only)
P21 `trust.follow-up-gaps`	Answers the next question too	**Bonus**	follow-up questions covered ÷ all. Not checked until the AI has run	`cq-followup-gaps` (was off)

- **P16 body and caching.** The body is `ParsedHtml.mainText`: the text of `<main>` (or `<body>` without header, nav and footer), one block per line, entities decoded, capped at 20,000 chars. Hidden elements are

skipped, except panels that a toggle controls (`aria-controls`), so collapsed FAQ answers are included (open item 3, fixed in Phase 4). In the panel it's the sanitised draft's main text, or live's when there's no draft; on the dashboard it's live's. `/api/ai-check` sends the first 3,000 chars to Claude on `ANTHROPIC_MODEL_BULK` (effort `low` , structured output). The panel calls `ai-check` once the body is ready and skips it when there's no text (console: `[scan] ai-check body: …`); see "Draft and live" for when it re-runs. Results are keyed by SHA-256 of the normalised title and text (`src/lib/ai/content-key.ts`) and cached in localStorage (`aiDiscovery:ai-results:<AI_CHECK_VERSION>` , shared by the panel and the dashboard) and in memory on the server, so an unchanged page doesn't call Claude again (console: `[ai-cache] hit …`). The AI re-run button skips both caches. A refusal returns `422` and the check stays not checked.

- **P17 author signals**, in order: the `author` of an Article-type JSON-LD node (a string, an inline Person/Organization, or an `@id` reference; linked when it has `url` or `sameAs`), then `<meta name="author">` , then a `rel="author"` link (which counts as the profile link).
- **P18 date sources**, in order: `<meta property="article:modified_time">` , a JSON-LD `dateModified` , `<meta name="last-modified">` , then Sitecore `__Updated` . `parseDate` (`helpers.ts`) reads ISO dates and Sitecore's compact `yyyyMMddTHHmmssZ` format.
- **P20 generic anchors.** `isGenericAnchor` (`helpers.ts`) matches the `ANCHOR_BLOCKLIST` exactly, plus a prefix rule limited to phrases that stay generic whatever follows ("click here …", "read more …", "see more …"). Descriptive anchors that start with a blocklisted word ("Download the 2026 report", "Register for the webinar") pass (SPIKE-FINDINGS §2).

`aeo-howto-schema` stays off (plan section 5.3); revisit only if HowTo content templates exist.

Site checks

Run once per site by `runSiteChecks` (`engine.ts`), with their own site score from the same pool maths (not shown since Phase 5). The dashboard lists them on the **Agent access** tab, and the ones that need work lead its Fix first list. S7 reads the home page's live HTML; the others grade the `/api/agent-access` scan (`SiteContext.access` , `AgentAccessData` ; how it's collected is in PAGE-MATRIX-TECHNICAL, "Agent access tab"). With no scan yet they're not checked ("The site hasn't been scanned yet.").

Bot groups (`src/lib/agent-access/bots.ts`): **Search** OAI-SearchBot, Claude-SearchBot, PerplexityBot, Amazonbot · **User** ChatGPT-User, Claude-User, Perplexity-User · **Training** GPTBot, ClaudeBot, Google-Extended, Applebot-Extended, meta-externalagent. S1 and S2 grade only the 7 search and user agents; a blocked training agent is mentioned ("which is your choice and doesn't lower the score") and never costs credit.

ID	Title	Tier	Credit	Notes
S1 `reach.ai-bot-access`	AI agents can open your site	Essential	search and user agents that can read ÷ those requested; **fail** if any is blocked	live requests, compared with a browser (`botGrid` : blocked = 4xx/5xx; challenge page = challenge markers or < 20% of the browser's size). Agents robots.txt disallows aren't requested and are left to S2. Not checked when the home page fails for a browser too, or when robots.txt disallows every search and user agent. The details end with the caveat that requests come from our server
S2 `reach.robots-ai-rules`	robots.txt lets AI search in	Essential	search and user agents allowed on `/` ÷ 7; **fail** if any is disallowed	RFC 9309 matching (`robots.ts`). No robots.txt (4xx) → 1. A 5xx → 0 (crawlers treat it as "keep out"). Unreachable → not checked. Replaces the Phase 3 six-crawler version
S3 `read.markdown-site`	Your site can serve Markdown to agents	Essential	no Markdown for `Accept: text/markdown` → 0; otherwise 0.5 + ⅙ each for HTML for browsers, `Vary: Accept` , Markdown smaller than the HTML	`gradeNegotiation` (`markdown-negotiation.ts`), shared with P5. Markdown = 200 + `text/markdown` + a body that isn't HTML and has Markdown syntax. On the dashboard the message adds "Markdown on N of M scanned pages" (P5 across the scanned pages, `SiteContext.markdownPages`); the credit stays the home page's
S4 `reach.sitemap`	Sitemap is published and linked	Recommended	0.5 for 200 + `<urlset>` / `<sitemapindex>` , +0.25 for `<lastmod>` on ≥ 80% (+0.125 for some), +0.25 when robots.txt names it	for an index, the counts come from its first sitemap
S5 `read.llms-txt`	llms.txt is published	Recommended	0.5 `# title` on the first line, +0.15 `> summary` , +0.15 a `- link` list, +0.2 a `## When to use…` section	missing, or a web page served at `/llms.txt` (soft 404) → 0 as a **warning**, not a fail. Button: **Open generator**

ID	Title	Tier	Credit	Notes
S6 `trust.trust-pages`	About, contact and privacy pages exist	Recommended	⅓ each: linked from the home page (footer first, shortest path) and 200; privacy also ≥ 500 characters of main text	replaces the Phase 5 footer-links row; terms, security page and postal address (`extractFooterTrust`) are in the details, not scored. Owner: author
S7 `understand.org-schema`	Your organisation is described for AI	Recommended	see below	`aeo-organization-schema`
S8 `reach.not-found`	Missing pages return a real 404	Recommended	404 or 410 → 1, anything else → 0	a 200 is a soft 404 ("redirects to the home page" when it ends on `/`)
S9 `reach.content-signals`	Content-use signals are declared	Bonus	a `Content-Signal:` line → 1; none → 0 shown as **Not set**	contentsignals.org; the Files helper writes it
S10 `read.link-headers`	Discovery links in HTTP headers	Bonus	an RFC 8288 `Link` header on the home page with `rel` describedby, sitemap, service-desc, service-doc, api-catalog, agent-skills or llms-txt, an `alternate` of type `text/markdown` , or a URL to llms.txt or a sitemap → 1; otherwise 0, **Not set**	
S11 `read.agent-protocols`	Agent protocols published	Bonus	found ÷ 2, capped at 1	**App or API only**, N/A otherwise. MCP server card (`/.well-known/mcp/server-card.json` or `/.well-known/mcp.json`), A2A agent card (`agent-card.json` , legacy `agent.json`), agent skills index, API catalog (RFC 9727), OAuth discovery (RFC 9728 / 8414): each must be a JSON object, not an HTML page served with 200. WebMCP: `navigator.modelContext` in the home page's inline scripts

ID	Title	Tier	Credit	Notes
S12 `read.commerce-protocols`	Agentic checkout protocols	Bonus	any found → 1	**Commerce only**, N/A otherwise. UCP (`/.well-known/ucp`), ACP (`/.well-known/acp.json`), MPP (`/openapi.json` with `x-payment-info`). x402 can't be detected from outside (it answers 402 on a paid request), so it's only noted

Paths for S11 and S12 were checked on 2026-11-02; MCP's server card path is still settling (SEP-2127), so both candidates are tried.

Real-site check (2026-11-02, BrandX demo site): robots.txt is `User-agent: * / Disallow: /`, so S2 fails and S1 isn't checked (no agent was requested); the home page returns HTML for `Accept: text/markdown` (S3: 0); a 177-URL sitemap with lastmod, listed in robots.txt (S4: 1); `/llms.txt`, the random path and every well-known path return a 200 HTML page (S5: warning, S8: soft 404, S11/S12: nothing found); about, contact and privacy all 200 (S6: 1); no Content-Signal or Link header. 22 requests in 1–3 s.

Real-site check with live bot requests (2026-11-02, the BrandX replica at hztl-page-health-checker-demo.vercel.app): robots.txt allows GPTBot, OAI-SearchBot, PerplexityBot, Google-Extended and ClaudeBot and disallows everyone else. The four with a crawler each got 200 on the home page, `/services` and `/contact-and-support`, the same size as the browser's, so S1 passes (1.0); S2 fails at 0.29 (2 of 7 search and user agents allowed); meta-externalagent and Applebot-Extended are an opt-out that costs nothing. The other results match the Sitecore site (S3 0, S5 warning, S6 1, S8 soft 404), and S7 is a warning because the Organization schema has no `contactPoint`. Its robots.txt lists the Sitecore host's sitemap first and its own second; S4 reads the one on the site's own origin (149 URLs, all with lastmod). 34 requests, 12 of them as bots, in 1.4–3.1 s. After the replica's robots.txt was changed to allow all seven search and user agents, opt the five training agents out and add `Content-Signal: search=yes, ai-input=yes, ai-train=no`: 43 requests, 21 as bots (each 200 on all three pages), in 3.1 s; S1, S2, S4, S6 and S9 pass, and the grid shows the five training agents as "Opted out (your choice)".

S7 `understand.org-schema` (`site/org-schema.ts`)

Graded on the **home page's live HTML** (the `getPageHTML` draft has no `<head>` scripts). The four rules agreed on 2026-10-31:

What the home page has	Result
Complete Organization schema	credit 1, pass
Organization schema with missing fields	partial credit, warning listing the fields, corrected snippet
No Organization schema (page fetched)	credit 0, **warning** ("Not added yet"), ready-to-paste snippet
Home page couldn't be fetched	**N/A** ("Couldn't check"), snippet

- **Credit:** 70% for the required fields (`name` ; `url` on the site's own domain, `www.` and subdomains allowed; `logo` as a URL or ImageObject) and 30% for the recommended ones (`sameAs` , `description` , `contactPoint`). Bonus fields (`address` as a PostalAddress, `legalName`) are listed as tips and not scored.
- **Detection:** every `application/ld+json` block, including `@graph` bundles, `@type` arrays (`["Organization", "LocalBusiness"]`), all 167 Organization subtypes from schema.org V30.1 (`schema-org-types.ts` , generated by `scripts/gen-schema-org-types.mjs`), and a `WebSite.publisher` that points to an Organization inline or by `@id` . The same entity declared twice under one `@id` counts once.
- **A block that doesn't parse** is a fail (credit 0) with the parse error shown.
- **Config is the expected value:** when `NEXT_PUBLIC_ORG_NAME` is set and the schema names a different organisation, or there are several Organization entries with conflicting names or URLs, the result is a warning and the credit is capped at 0.9.
- **Fix snippet:** starts from the schema already on the site and fills only the gaps (name, url, logo, description, sameAs) from `NEXT_PUBLIC_ORG_*` (`getOrganizationInfo` , with the site's name and domain as fallbacks). A `url` off the site's domain is replaced; other values are kept. With no schema at all it's `buildOrganizationJsonLd` .
- **In the panel** S7 shows only when the home page is open, in a "Site check — not in this page's score" section, graded on the open page's live HTML (so it follows a re-scan).

Site issues in the panel (Phase 6)

The panel reads the same scan through `useAgentAccess` (cached an hour per domain, so moving between pages doesn't rescan the site; the panel's Re-scan doesn't re-run it) and runs `runSiteChecks` with it. `SiteIssues` (`src/components/panel/SiteIssues.tsx`), between the category bars and Top fixes, shows the failing **Essential Reach** site checks, S1 and S2: the ones that keep agents out of every page. It's a red strip, "N site issue(s) affect(s) every page:" with each check's message and **View in dashboard**, which opens `/dashboard-widget-extension#agent-access` (`?demo=1` kept in demo mode) through `navigateToExternalUrl` , or a new tab. Other site checks (Markdown, llms.txt, …) stay on the dashboard, so the panel isn't permanently red for issues an author can't act on; Markdown is scored per page as P5 instead. No site check is part of the page score.

- **Evidence:** the BrandX home page has no JSON-LD at all (checked 2026-10-31 and 2026-11-01). The old check gave it 5/5; S7 gives it 0 with a warning and a snippet.

Shared JSON-LD parser (`json-ld.ts`)

`extract-html.ts` keeps every block as `ParsedHtml.jsonLdScripts` , either `{ ok: true, value }` or `{ ok: false, error }` , so parse errors are reported instead of dropped. `collectNodes` turns the blocks into typed nodes: each top-level object, each array entry and each `@graph` member (nested objects stay inside their parent; read them with `resolve` , which also follows `{"@id": …}` references). `@type` is always a list with `schema:` / `https://schema.org/` prefixes removed. `isOrganization` and `isArticle` match subtypes.

The view model and the panel UI (Phase 5)

`view-model.ts` turns reports into what the screens show. It never changes a score: everything comes from the points the engine computed.

- **Rows:** `toCheckRow` adds `display` (`pass` / `improve` / `fix` / `na` / `not-checked` , from `state` and `status`), `gain` (`maxPoints - points`) and `gainPts` (rounded, for "+N pts"; 0 hides it), plus the check's `why` , `owner` and `fix` . `STATUS_TEXT` , `OWNER_TEXT` and `SOURCE_TEXT` hold the words.
- **Top fixes:** `rankedFixes` keeps Improve and Fix rows worth at least 1 point, sorted by gain, then Essential first, then author fixes first. `topFixes` takes three.
- **Verdict:** `pageVerdict` gives the label from `labelForPct` and one sentence. `fixesToReady` adds the biggest gains to the score until it reaches 80: "N fixes would make this page agent-ready". If even every fix wouldn't, it says how many checks need work; at 80+ it says how many could still improve.
- **Fix buttons:** `fixButton` : **Generate** for P9 and P11 (the generator's fields; other `generate` fixes wait for Phase 9), **Copy snippet** for a `copy-snippet` fix with a `codePreview` , **Send to developer** for developer-owned fixes (copies `developerNote` : title, page URL, why, what we found, fix, details, code), otherwise **Show how** (opens and scrolls to the row). Copying uses `src/lib/clipboard.ts` , which falls back to `execCommand` because the Marketplace iframe doesn't delegate `clipboard-write` .
- **Panel** (`src/app/pages-contextpanel-extension/page.tsx` , `src/components/panel/`): header with the Draft/Live chip (Draft when `getPageHTML` gave a draft) and Re-scan; Radix tabs Score / Agent View (`AgentViewTab` , below) / AI Assist (`AssistTab` , below). The tabs are force-mounted, so AI Assist keeps its suggestions when you switch tabs. `ScoreTab` holds the banner, `VerdictCard` , `CategoryBars` (toggle buttons that filter the list), `TopFixes` and the check groups (`CheckRow`), with a Needs work / All switch. Needs work also lists not-checked AI and PageSpeed rows, which have a **Run** button. N/A rows are listed under "Doesn't apply to this page". On the home page a Site check group shows S7, and a red site-issues strip (S1, S2) sits above Top fixes when agents are blocked site-wide.
- **Agent View** (`src/components/panel/AgentViewTab.tsx` , mockup `PanelAgentView`): from `useMarkdownCheck` (`/api/markdown-check` on page change and Re-scan; a Re-scan keeps the last result until the new one is in, and rows show "Scanning…" while it loads, since P5 is Essential). Top to bottom:

- "What AI agents receive" and one line (sends Markdown / some setup missing / sends the full HTML).

- Four status tiles from `negotiationTiles` (`markdown-negotiation.ts`): Markdown response, HTML fallback, Vary: Accept, Distinct versions (grey "Checked once Markdown is served" without Markdown).

- The token bar: "Markdown saves N% fewer tokens" (measured) or "If you served Markdown: about N% fewer tokens" (HTML tokens vs the preview's).

- **Compare**, with a **Before / After** switch (Phase 13; it replaced the mockup's Human / Agent). It opens on Before when the page doesn't serve Markdown and on After when it does; a pick holds until the page URL changes.

- **Before** ("Without optimisation: the full web page an agent downloads today."): a stacked bar of `html.anatomy` and a legend in words with tokens and shares (content in primary, the four waste kinds, `WASTE_KINDS`, in four reds, structured data grey; a share under 0.5% reads "under 1%"); "Your content is N% of what an agent downloads. M% (T tokens) is code and layout." (M counts only the red kinds, not JSON-LD); then the excerpt in a `<pre tabIndex={0}>` that wraps anywhere, each red run in a red `<mark>` and JSON-LD in a grey one, each with the category as a `title` and in a visually hidden label; "Showing the first N of M tokens.", plus "Your content starts about N tokens in." when no body content is in the excerpt. Without an anatomy, the HTML's token count only.

- **After** ("With optimisation: the same page as Markdown."): the served Markdown, or the preview converted from the draft (from the route's conversion of the published page without a draft), labelled "Preview: your site doesn't serve this yet"; then **What was removed**: each red kind with tokens struck through (`<del>`) and its tokens in green, and the total, HTML tokens minus Markdown tokens (the token bar's saving; measured when served, "about" otherwise). The rows don't add up to the total exactly: Markdown adds front matter, link URLs and syntax, and drops JSON-LD.

- **Why it matters**, in both views: it fits (this page's tokens, and how many are content), it's easier to quote, it's cheaper to read (the Markdown's tokens against the HTML's). No claims about specific AI vendors.

- "Is the Markdown complete?": P6's three parts as rows, from `gradeMarkdownQuality`; a grey "Checked once your site serves Markdown" until then.

- "How to fix" (Developer) unless all four pass: `MarkdownFixes` (`src/components/shared/MarkdownFixes.tsx`; accept-md with Copy, Cloudflare Markdown for Agents, your own route behind Show code; content in `src/lib/markdown/fixes.ts`) and **Send to developer** with P5's note.

- Loading, request failed and "published page wasn't found" states.

- **"▲ N since last scan"** (`useScanDelta`, `src/hooks/useScoreHistory.ts`): when a scan settles (the page, its draft and the AI and PageSpeed checks have loaded), the stored score for the page (`aiDiscovery:pageScore:v1:<pageId>|<language>`) becomes the baseline and the new score is stored. Draft edits afterwards show against that baseline. Demo mode uses a fixed baseline (`DEMO_PREVIOUS_PAGE_SCORE`).
- **Shared components** (`src/components/shared/`): `StatusIcon` (icon, colour and word; `@mdi/js` paths), `ScoreRing` (fills on first render), `VerdictCard`, `CategoryBars`, `TopFixes`, `CheckRow`, `FixButton`, `Segmented`, `EmptyState`, the chips (`OwnerBadge`, `GainChip`, `SourceChip`, `DeltaChip`), `AppMark`.

Blok's components are in `src/components/ui/` and its theme tokens in `globals.css` (Sitecore/blok @ 0a7997e, light theme only).

- **Accessibility:** every status has an icon and a word; buttons and tabs are keyboard-reachable with a visible focus ring; the tab panels are focusable; the panel works at 320 px (the source chip wraps under the title). Motion is off under `prefers-reduced-motion`.
- **Dashboard:** `summarisePageReport` → `PageScoreSummary` (adds `issues` for Fix first) and `groupPageIssues` / `siteIssueGroups` / `issueNote`. See PAGE-MATRIX-TECHNICAL.

AI Assist and write-back (Phase 9)

The panel's AI Assist tab (`src/components/panel/AssistTab.tsx` , cards in `src/components/panel/assist/`) replaces `MetaGenerator`. Each card shows "Now: …" and an editable suggestion, then **Apply** (through the confirm dialog) or **Copy**. Cards are ordered by the gain of their check (`gainPts`); the summary card comes last. **Generate** on a Score-tab row passes the check ID (`onGenerate(checkId)`), so the tab opens scrolled to that card. P12, P13 and P15 now have `fix: "generate"` ; P17 has `fix: "apply-field"` , shown as **Choose author** (`fixButton` in `view-model.ts`).

Card	Check	AI	Writes to	Call
Title, description and social	P9, P11	`/api/generate-meta`, effort low	the page's `pageTitle`, `MetaDescription`, `OpenGraphTitle`, `OpenGraphDescription`	`xmc.pages.updateFields`
Quotable FAQ answers	P12	`/api/assist` `faq-answers`, low	the AccordionItem datasource's `content` (rich text; the `ck-content` wrapper is kept)	authoring `updateItem`
Image alt text	P15	`/api/assist` `alt` (vision, image by URL), low	`alt` on the image field of the page item or the component datasource; never the media item	`updateFields` (page) or `updateItem` (datasource)
Author byline	P17	none	the page's `authorName`	`updateFields`
Structured data	P10	none: P10's `codePreview`	the field in `NEXT_PUBLIC_FIELD_JSON_LD` when set and on the template; copy-only otherwise (BrandX has none)	`updateFields`
Question headings	P13	`/api/assist` `question-headings`, medium	copy-only	—
Summary paragraph	—	`/api/assist` `summary`, low	copy-only	—

Writes (`src/lib/sitecore/write.ts`; findings in `SPIKE-FINDINGS.md` section 9):

- `writeField(client, contextId, request)` is the only write path: `updateFields` for page fields, and `updateItem(input: { itemId, language, fields: [{ name, value }] })` (with `$id: ID!`) for datasource items. Both write the current version in place, in the page's language, and never publish. Item IDs go out in `normalizeItemId` form.
- **Any GraphQL `errors` entry is a failed write**, even with data alongside. A host refusal (`isHostAuthError`) shows `HOST_AUTH_WRITE_MESSAGE`.
- The value is read back (`updateFields` returns no field values; `updateItem` returns the item). A value that didn't stick is a failure.
- **Audit:** one console line per write, success or not (`auditLine`): `[audit] wrote: <label> on <item> (<id>, field "<name>", <language>, <call>): "<before>" → "<after>", version N, M ms, not published`, with the full request as a second argument. The UI adds a toast (Blok sonner).
- **Confirm:** `WriteConfirmProvider` and `useConfirmWrite()` (`src/components/shared/WriteConfirm.tsx`, Blok alert-dialog) list each change: label, item, field, language, before → after (`display` holds the readable text when the raw value is markup), and a shared-component warning. Only **Write to Sitecore** sends. It's an in-page dialog because Page Builder's iframe blocks `window.confirm`.

- **Why Apply is off** (`writeDisabledReason`), shown next to the button: no Marketplace client (except in demo mode); no field configured; the item's template doesn't have the field; `workflow.canEdit` is false.
- **Shared components:** a datasource outside the page's own item path (e.g. `Data/Feature/Home - Advisor`) is shared (`isSharedDatasource`); the confirm says the change shows on every page that uses it.
- After a successful write the panel re-reads the fields (`usePageFields().refetch`) and the draft (`usePageHtml().refresh`).

Field mapping (`src/lib/sitecore/field-map.ts` , `FIELD_MAP`): every field name the app reads or writes, each overridable with a `NEXT_PUBLIC_FIELD_*` variable (`TITLE` , `META_DESCRIPTION` , `OG_TITLE` , `OG_DESCRIPTION` , `AUTHOR` , `JSON_LD` , `FAQ_QUESTION` , `FAQ_ANSWER`). `SCORING_FIELD_NAMES` and `usePageFields` read it. `PageWriteContext.fieldNames` holds each field's name as the page item spells it, or null when the template lacks it.

Where the data comes from:

- **FAQ:** `usePageFields` keeps each AccordionItem pair with its datasource item (`faqSources` : item ID, path, question, raw answer, answer field name), which settles plan section 13, item 4. Without datasource pairs the card uses the draft's Q&A pairs, copy-only.
- **Images** (`src/lib/assist/images.ts` ; `useImageFields` in `src/hooks/useAssistSources.ts` , run when the tab opens): the image fields on the page item and on every datasource in the page's layout (one aliased GraphQL query), then their media paths (`mediaItem(where: { mediaItemId })` , one aliased query). A draft `<img src>` (`…/-/media/<mediaPath>.jpg`) is matched to its field by media path; images in rich text, or with no field, are copy-only. `altStatus` flags a missing alt and a file-name alt (no spaces with `-` or `_` , the file's own name, `IMG_123`).
- **Authors** (`useAuthorNames`): the distinct `authorName` values on the page's sibling items. BrandX has no Authors folder (SPIKE-FINDINGS section 9.3).

`/api/assist` (`src/lib/server/assist.ts`): `{ kind, pageTitle, mainText, … }` , validated with zod, answers `AssistData` . The brand voice (system) and the page text (first user block) are cache breakpoints, and the task follows them. Structured outputs, `fallbacks: "default"` , effort low (medium for question headings). Results are cached in server memory by SHA-256 of the inputs (`ASSIST_VERSION` , kind, model, text, inputs); a hit answers with `cached: true` .

- **Alt text** sends the image as `{ type: "image", source: { type: "url" } }` (Sitecore media URLs get `mw=1024`). If Claude can't fetch it (`BadRequestError` , e.g. a CM media URL behind a login), it retries from the file name and the page text and returns `sawImage: false` , and the card says so.
- FAQ answers are checked against 40–300 characters (`inRange`), alt text against 125 (`overLimit`).

`/api/generate-meta` (`src/lib/server/generate-meta.ts`): `{ field, pageTitle, bodyText, currentValue }` answers `{ value, maxChars, overLimit }` , unchanged for the panel. The dashboard sends `pageUrl` instead of `bodyText` : the route reads the published page through `safeFetch` (`readPageForQuestions`) and caches the result by content hash (`cached: true` on a hit). The panel's Regenerate always asks again.

- **Model:** `ANTHROPIC_MODEL` (default `claude-opus-5-5`), effort `low` , structured output `{ value }` .

- **Length limits** (title 60, description 155, OG title 70, OG description 200): an overrun gets one retry with the count fed back; still over gives `overLimit: true` and an amber count. Values are never cut off.
- **Prompt caching:** the brand voice and the page text carry `cache_control`, and the field instruction comes after them. The server log shows `cache_read=N` on each `[anthropic] generate-meta:<field>` line.
- **Brand voice:** `getBrandVoice()` (`src/lib/brand/brand-voice.ts`) reads `BRAND_VOICE_FILE` and `BRAND_NAME`, else a neutral default. `brand/brandx.md` is the BrandX demo voice. The SitecoreAI brand kit stays out of scope (plan section 11).

Demo mode (`?demo=1`): `makeTestAssist()` (`test-fixture.ts`) holds a suggestion for every card, the page's fields, FAQ datasources, three image fields (a page field, a page-local component and a shared component) and author names. `demoAnswers()` (`src/lib/assist/demo.ts`) answers `requestAssist` and `requestMeta` with no request, and image previews aren't loaded. The demo page is a BrandX article, "Retirement planning in your 40s". Apply works as far as the confirm dialog: `WriteConfirmProvider` gets `demo`, and **Write to Sitecore** then logs `[audit] demo, nothing written: …` and shows the toast "Demo: nothing was written to Sitecore." with no request (there's no Marketplace client in demo mode). The field checks still apply, so the JSON-LD card stays copy-only and the image in rich text shows Copy, as on BrandX. The Symposium walk-through is `docs/SYMPOSIUM-DEMO-SCRIPT.md`.

Server-side fetching (SSRF protection)

Every server route that fetches a caller-supplied URL (`preview-fetch`, `agent-access`, `markdown-check`, `citation-probe` for the page it suggests questions from, and `psi-check` for its Google call) uses `safeFetch` from `src/lib/server/safe-fetch.ts`. Phase 6 removed `robots-check` and `footer-check`; `agent-access` does both.

- `http` / `https` only; no credentials in the URL.
- Blocks private, loopback, link-local (incl. `169.254.169.254`), CGNAT, multicast, reserved and documentation ranges, IPv4 and IPv6, including IPv4-mapped and NAT64/6to4/Teredo forms. The check runs in the undici Agent's DNS lookup at connect time, so DNS rebinding can't bypass it. IP-literal hosts (including `http://2130706433/`-style encodings) are checked before the request.
- Follows at most 3 redirects (re-checked each hop) and drops the caller's headers, such as the editing token, on a cross-origin hop. `publicHeaders` (User-Agent, Accept: nothing secret) are sent on every hop, so a bot request stays a bot request after `example.com` → `www.example.com`.
- One timeout across redirects and body; bodies capped at 5 MB by default. An optional `signal` ends it early (the agent-access deadline), reported as a timeout.
- The returned `Response.url` is the final URL after redirects.
- Blocked input → `SafeFetchError` with `isBadInput`, and the route answers **400** `{ error: "blocked_url" }`. Fetch failures stay **502**, or the route's own fallback.
- `SAFE_FETCH_ALLOW_HOSTS` (comma-separated) skips the address check for named hosts. Use it only for local development.

- Tests: `src/lib/server/safe-fetch.test.ts`.

Tests

File	Covers
`src/lib/checks/scoring.test.ts`	pools, partial credit, weights, N/A and not checked, rescaling, bonus cap, categories, the gate
`src/lib/checks/page/page-checks.test.ts`	P10 real detection (D2), P9, N/A and not-checked cases, P17, P18, P3 (Related Articles, edits, empty states), body vs head sources, source chips, the fixture page and draft
`src/lib/checks/draft-diff.test.ts`	block comparison (client-only, edits, tokens, split blocks) and the "Draft differs" rule
`src/hooks/usePageHtml.test.ts`	`readWithRetry` : the first read retried while the canvas loads, giving up, no retry with a draft, a newer read taking over
`src/lib/html/sanitise-draft.test.ts`	the sanitiser on the BrandX FAQ, Banking article and St. Patrick's Day captures (`src/lib/html/fixtures/`), plus help text, Fallback hints, `[No text in field]` , `scEmpty*` placeholders, empty states, tokens, editor attributes
`src/lib/checks/brandx-pages.test.ts`	the whole pipeline on real BrandX draft + live pairs: P3 on the Related Articles cards, the footer token, the banner, P15 and P20
`src/lib/checks/site/org-schema.test.ts`	S7: the four rules, `@graph` , `@type` arrays, subtypes, `WebSite.publisher` , parse errors, config mismatch, duplicates, BrandX
`src/lib/checks/site/agent-access.test.ts`	S1–S12 on the demo scan (ChatGPT-User 403: the Phase 6 accept-when case) and variants: challenge and stub pages, sample-page blocks, training agents, timeouts, robots.txt states, Markdown negotiation, sitemap, llms.txt parts, trust pages, soft 404, Content-Signal, Link headers, S11/S12 by site type
`src/lib/agent-access/robots.test.ts`	RFC 9309 groups, `*` fallback, longest match, `*` and `$` , versioned tokens, Content-Signal, the robots.txt helper's lines
`src/lib/server/agent-access.test.ts`	the collector against a local server (a firewall returning 403 to ChatGPT-User, GPTBot never requested, samples, sitemap, trust pages, well-known JSON vs HTML), the deadline, private addresses, the link pickers
`src/lib/server/markdown-check.test.ts`	the collector against a local server: both Accept headers, a served Markdown body without a preview, the preview of an HTML-only page, the token breakdown with and without Markdown, the excerpt only when asked for, a missing page, private addresses

File	Covers
`src/lib/markdown/anatomy.test.ts`	the breakdown on the BrandX captures (the categories add up to the page's tokens, content close to `mainText`, JSON-LD as structured data, collapsed FAQ answers as content), each category on a small page, a page without `<main>` or `<body>`, the excerpt (the page's first characters, merged runs, not cut when short), where the content starts, and the demo page (under 5% content, over 90% waste, its content inside the excerpt, the same breakdown with and without Markdown)
`src/lib/markdown/markdown.test.ts`	the conversion (main content, chrome, scripts, collapsed answers, absolute links, front matter quoting), P6's grader, P5 and P6 on the demo page (4/4 served, and not served with the estimated saving: the Phase 7 accept-when cases), partial credit, not checked, S3's page count, Markdown once in Fix first
`src/lib/checks/json-ld.test.ts`	the shared parser
`src/lib/checks/view-model.test.ts`	status words, Top fixes ranking, "N fixes would make this page agent-ready", fix buttons, the developer note, page issues in the summary, Fix first grouping
`src/lib/llms-txt/build.test.ts`	the "When to use this site" section and the score filter
`src/lib/relative-time.test.ts`	"N min ago" / "N h ago" / "N days ago"
`src/lib/checks/parity.test.ts`	panel and dashboard read the same fields and give the same score on the same inputs; a draft changes the panel's; the 80 / 60 thresholds
`src/lib/html/extract-html.test.ts`	`mainText` (collapsed answers included), entity decoding, `bodyBlocks`, Q&A pairs (BrandX accordion markup), metadata, JSON-LD blocks

Key files

File	Responsibility
`src/lib/checks/registry.ts`	Every check, in display order
`src/lib/checks/page/{reach,read,understand,trust}.ts`	Page checks P2–P21
`src/lib/checks/site/{reach,read,trust,org-schema}.ts`	Site checks S1–S12
`src/lib/checks/markdown-negotiation.ts`	The accept.md checks (S3, P5) and the Agent View's status tiles
`src/lib/server/markdown-check.ts`, `src/app/api/markdown-check/route.ts`, `src/hooks/useMarkdownCheck.ts`	The page as an agent and as a browser (P5, P6)
`src/lib/markdown/anatomy.ts`	Where the HTML's tokens go, and its classified first 16 KB (the Agent View's Before view)
`src/lib/markdown/`	The preview conversion and token estimate, P6's grader, the fix cards' content, the route's types
`src/components/panel/AgentViewTab.tsx`, `src/components/shared/MarkdownFixes.tsx`	The Agent View tab and the fix cards (also under S3 on the dashboard)
`src/lib/agent-access/`, `src/lib/server/agent-access.ts`, `src/app/api/agent-access/route.ts`	Bots, robots.txt, the bot grid and the scan behind the site checks
`src/lib/checks/scoring.ts`, `gate.ts`, `engine.ts`	Pool maths, the gate, `runPageChecks` / `runSiteChecks`
`src/lib/checks/context.ts`	`buildPageContext` (draft → `body`), the shared field names, article/home detection
`src/lib/checks/draft-diff.ts`	Draft vs live block comparison: P3 and the "Draft differs" banner
`src/hooks/usePageHtml.ts`	Reads the draft with `getPageHTML`, debounced on field updates, retried while the canvas loads, with the live fallback
`src/components/panel/`	Score tab, the draft banner, the gate banner, the scan note
`src/components/shared/`	Status icon, score ring, verdict card, category bars, Top fixes, check row, fix button, chips
`src/hooks/useScoreHistory.ts`	"Since last scan" (panel) and the last 8 site scores (dashboard)

File	Responsibility
`src/lib/checks/json-ld.ts`, `schema-org-types.ts`	JSON-LD parsing and the schema.org subtype lists
`src/lib/checks/view-model.ts`	Reports → rows, Top fixes, verdicts, fix buttons, developer notes, dashboard summaries and Fix first
`src/lib/sitecore/write.ts`, `field-map.ts`	Every write to Sitecore (confirmed in the UI first), the audit line, why Apply is off, image-field and rich-text values; the field names
`src/components/shared/WriteConfirm.tsx`	The confirm dialog (before → after) and the toast, for the panel and the dashboard
`src/components/panel/AssistTab.tsx`, `src/components/panel/assist/`	The AI Assist tab and its cards
`src/lib/assist/`, `src/hooks/useAssistSources.ts`	Image-to-field matching, the browser calls (and their demo answers), the image fields and author names
`src/lib/server/assist.ts`, `src/app/api/assist/route.ts`	Alt text, FAQ answers, question headings and the summary (Claude)
`src/lib/rule-engine/helpers.ts`	ARI, word counts, anchor blocklist, dates, snippet builders
`src/lib/rule-engine/types.ts`	Input types (`ParsedHtml`, `AiResults`, `PageData`, …)
`src/lib/rule-engine/test-fixture.ts`	Demo data for `?demo=1`, including the panel's demo draft (`makeTestDraft` : one edited heading, client-rendered Related Articles cards). Demo mode never starts the Marketplace client and doesn't auto-scan, so it makes no SitecoreAI or paid calls on load (`useDemoMode.ts`, `useMarketplaceClient.ts`)
`src/lib/score-thresholds.ts`	Shared 80 / 60 thresholds
`src/lib/html/extract-html.ts`	Parses HTML into `ParsedHtml` (server for live, browser for the draft)
`src/lib/html/sanitise-draft.ts`	Removes Page Builder artifacts from the draft before parsing
`src/lib/brand/organization.ts`, `brand-voice.ts`	Organization identity and brand voice config